F Y E O

Security Code Review of the Natgold Presale Backend

Natgold

February 2026
Version 1.0

Presented by:
FYEO Inc.
PO Box 147044
Lakewood CO 80214
United States

Security Level
Public

Table of Contents

Executive Summary....2
- Overview....2
- Key Findings....2
- Scope and Rules of Engagement....3

Technical Analyses and Findings....4
- Findings....5
- The Classification of vulnerabilities....5
- Technical Analysis....6
- Conclusion....6

Technical Findings....7
- General Observations....7
- Chainalysis API DoS Vulnerability....8
- Session Cleanup Task Failure Can Crash Entire Server....9
- Connect Wallet Lacks Database Transaction....10
- Missing Zero Address Checks in Initialize....12
- PAC Endpoint Ignores Expiration and Hardcodes KYC Status....13
- The fund() Function Lacks Access Control....14
- Token Decimals Not Verified....15

Our Process....16
- Methodology....16
- Kickoff....16
- Ramp-up....16
- Review....17
- Code Safety....17
- Technical Specification Matching....17
- Reporting....18
- Verify....18

Additional Note....18

Executive Summary

Overview

Natgold engaged FYEO Inc. to perform a Security Code Review of the Natgold Presale Backend.

The assessment was conducted remotely by the FYEO Security Team. Testing took place on February 05 - February 09, 2026, and focused on the following objectives:

- To provide the customer with an assessment of their overall security posture and any risks that were discovered within the environment during the engagement.
- To provide a professional opinion on the maturity, adequacy, and efficiency of the security measures that are in place.
- To identify potential issues and include improvement recommendations based on the results of our tests.

This report summarizes the engagement, tests performed, and findings. It also contains detailed descriptions of the discovered vulnerabilities, steps the FYEO Security Team took to identify and validate each issue, as well as any applicable recommendations for remediation.

Key Findings

The following issues have been identified during the testing period. These should be prioritized for remediation to reduce the risk they pose:

- FYEO-NATP-01 – Chainalysis API DoS Vulnerability
- FYEO-NATP-02 – Session Cleanup Task Failure Can Crash Entire Server
- FYEO-NATP-03 – Connect Wallet Lacks Database Transaction
- FYEO-NATP-04 – Missing Zero Address Checks in Initialize
- FYEO-NATP-05 – PAC Endpoint Ignores Expiration and Hardcodes KYC Status
- FYEO-NATP-06 – The fund() Function Lacks Access Control
- FYEO-NATP-07 – Token Decimals Not Verified

Based on our review process, we conclude that the reviewed code implements the documented functionality.

Scope and Rules of Engagement

The FYEO Review Team performed a Security Code Review of the Natgold Presale. The following table documents the targets in scope for the engagement. No additional systems or resources were in scope for this assessment.

The source code was supplied through a private repository at https://github.com/fpco/natgold-presale with the commit hash 2959120cd6ec3c39148f4f5a9906d368ec28b882.

Remediations were submitted with the commit hash db200500576f52da813d6f2a564ac9e90dfe7da1.

Files included in the code review

`natgold-presale/`
`├── backend/`
`│  ├── kyc-token/`
`│  │  └── src/`
`│  │  ├── lib.rs`
`│  │  └── main.rs`
`│  └── natgold-presale/`
`│  └── src/`
`│  ├── eth/`
`│  │  └── signature.rs`
`│  ├── chainalysis.rs`
`│  ├── eth.rs`
`│  ├── lib.rs`
`│  └── main.rs`
`└── contract/`
`  ├── contracts/`
`  │  ├── NatGoldPresale.sol`
`  │  ├── Token.sol`
`  │  └── UsdtToken.sol`
`  ├── scripts/`
`  │  └── send-op-tx.ts`
`  └── hardhat.config.ts`

Table 1: Scope

Technical Analyses and Findings

During the Security Code Review of the Natgold Presale, we discovered:

- 1 finding with HIGH severity rating.
- 1 finding with LOW severity rating.
- 5 findings with INFORMATIONAL severity rating.

The following chart displays the findings by severity.

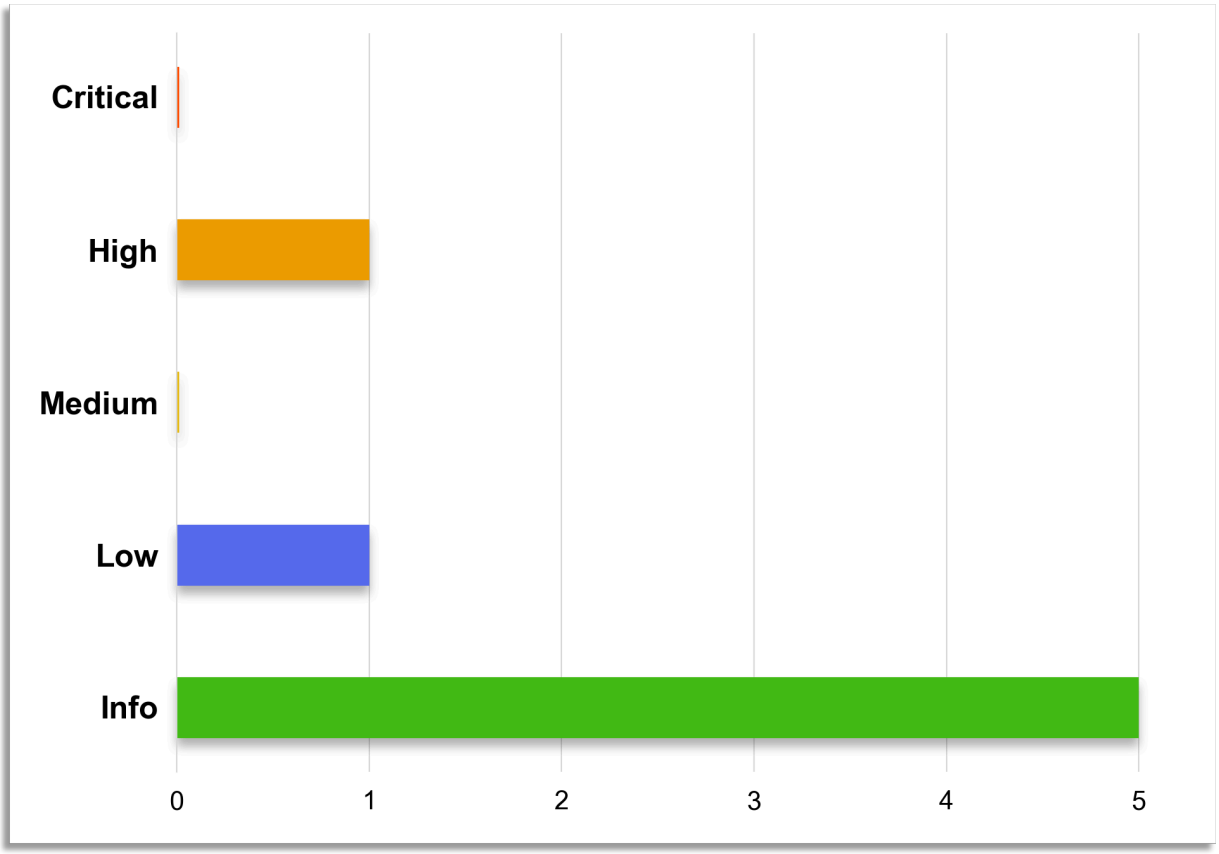

Figure 1: Findings by Severity

Findings

The *Findings* section provides detailed information on each of the findings, including methods of discovery, explanation of severity determination, recommendations, and applicable references.

The following table provides an overview of the findings.

Finding #	Severity	Description	Status
FYEO-NATP-01	High	Chainalysis API DoS Vulnerability	Remediated
FYEO-NATP-02	Low	Session Cleanup Task Failure Can Crash Entire Server	Remediated
FYEO-NATP-03	Informational	Connect Wallet Lacks Database Transaction	Remediated
FYEO-NATP-04	Informational	Missing Zero Address Checks in Initialize	Remediated
FYEO-NATP-05	Informational	PAC Endpoint Ignores Expiration and Hardcodes KYC Status	Acknowledged
FYEO-NATP-06	Informational	The fund() Function Lacks Access Control	Remediated
FYEO-NATP-07	Informational	Token Decimals Not Verified	Remediated

Table 2: Findings Overview

The Classification of vulnerabilities

Security vulnerabilities and areas for improvement are weighted into one of several categories using, but is not limited to, the criteria listed below:

Critical – vulnerability will lead to a loss of protected assets

- This is a vulnerability that would lead to immediate loss of protected assets

- The complexity to exploit is low
- The probability of exploit is high

High - vulnerability has potential to lead to a loss of protected assets

- All discrepancies found where there is a security claim made in the documentation that cannot be found in the code
- All mismatches from the stated and actual functionality
- Unprotected key material
- Weak encryption of keys
- Badly generated key materials
- Txn signatures not verified
- Spending of funds through logic errors
- Calculation errors overflows and underflows

Medium - vulnerability hampers the uptime of the system or can lead to other problems

- Insecure calls to third party libraries
- Use of untested or nonstandard or non-peer-reviewed crypto functions
- Program crashes, leaves core dumps or writes sensitive data to log files

Low – vulnerability has a security impact but does not directly affect the protected assets

- Overly complex functions
- Unchecked return values from 3rd party libraries that could alter the execution flow

Informational

- General recommendations

Technical Analysis

The source code has been manually validated to the extent that the state of the repository allowed. The validation includes confirming that the code correctly implements the intended functionality.

Conclusion

Based on our review process, we conclude that the code implements the documented functionality to the extent of the reviewed code.

Technical Findings

General Observations

The NatGold presale system is a two-part application that allows users to purchase NatGold tokens using USDT while ensuring compliance with financial regulations. The backend server acts as a gatekeeper, verifying that users have completed Know Your Customer (KYC) checks and are not on sanctions lists before allowing them to participate. When a user wants to buy tokens, they first connect their cryptocurrency wallet to the backend, which checks their address against Chainalysis (a blockchain compliance service) to ensure they're not sanctioned. If they pass this screening and have approved KYC status, the backend issues them a cryptographically signed "Purchase Allowance Claim" (PAC) - essentially a digital permission slip that proves they're authorized to participate in the sale. This PAC is valid for 24 hours and can be refreshed as needed.

The smart contract on the Ethereum blockchain handles the actual token sale mechanics. Users submit their PAC along with their purchase transaction, and the contract verifies the PAC's signature to confirm it was issued by the authorized backend server and that it matches the buyer's wallet address. The sale happens in phases: first, users deposit USDT and specify how much they want to buy; then administrators run a settlement process that calculates how many tokens each user receives based on available supply and pricing (including discounts for early participants); finally, the contract is funded with NatGold tokens and users can claim their purchased tokens along with any USDT refunds if their full amount couldn't be filled. Throughout this process, the backend maintains session management for users to easily refresh their PACs, while the smart contract ensures that only verified, non-sanctioned users can participate and that the token distribution is fair and transparent.

Chainalysis API DoS Vulnerability

Finding ID: FYEO-NATP-01
Severity: **High**
Status: **Remediated**

Description

The `/challenge` and `/connect-wallet` endpoints have no rate limiting and query the Chainalysis API on every connection attempt. An attacker can generate unlimited random wallets and valid signatures to exhaust the Chainalysis API quota (5000 requests per 5 minutes), preventing all legitimate users from connecting. Currently re-using the same address does also query the API again.

Proof of Issue

File name: backend/natgold-presale/src/lib.rs
Line number: 375

```
match state
    .chainalysis
    .check_address(&wallet_address_text)
    .await?
{
    ChainalysisCheck::Clear => {}
    ChainalysisCheck::Sanctioned => return Err(AppError::SanctionedAddress),
}
```

Severity and Impact Summary

Complete service denial. Attacker can block all users from connecting wallets and obtaining PAC tokens, effectively preventing any purchases. Attack is trivial to execute (generate random wallets, sign challenges) and costs the attacker nothing.

Recommendation

Implement IP-based rate limiting on `/challenge` and `/connect-wallet` endpoints. Consider implementing CAPTCHA for additional protection. Cache Chainalysis results per address with reasonable TTL to reduce API calls.

Remediation

Request/Rate limiting per IP has been added via AWS CloudFront. API limits have been increased.

Session Cleanup Task Failure Can Crash Entire Server

Finding ID: FYEO-NATP-02
Severity: **Low**
Status: **Remediated**

Description

The session cleanup background task runs alongside the main HTTP server in a `JoinSet`. If the cleanup task panics or encounters an unrecoverable error (database connection loss, pool exhaustion, network issues), the entire server process terminates because the `JoinSet` waits for the first task to complete and then aborts all others.

Proof of Issue

File name: backend/natgold-presale/src/lib.rs
Line number: 609

```
let mut tasks = JoinSet::new();
tasks.spawn(async move {
    info!("http server started");
    server.await.map_err(|_| "server error")
});
tasks.spawn(async move {
    info!("session cleanup task started");
    cleanup_expired_sessions(db)
        .await
        .map_err(|_| "cleanup error")
});

if let Some(result) = tasks.join_next().await {
    match result {
        Ok(Ok(_)) => info!("task exited"),
        Ok(Err(err)) => error!(%err, "task returned error"),
        Err(err) => error!(%err, "task panicked"),
    }
}
tasks.abort_all();
```

Severity and Impact Summary

Database connection issues or other transient errors in the cleanup task can cause the entire server to shut down, requiring manual restart. This affects service availability even though the HTTP server itself may be functioning correctly.

Recommendation

Isolate cleanup task failures from main server - use separate task spawning without `JoinSet` coupling. Implement health monitoring that alerts on cleanup failures without terminating the server.

Connect Wallet Lacks Database Transaction

Finding ID: FYEO-NATP-03
Severity: **Informational**
Status: **Remediated**

Description

The `connect_wallet` function performs two separate database operations (user account upsert and session creation) without wrapping them in a transaction. This creates a race condition window where concurrent requests could lead to inconsistent state, such as sessions being created for non-existent users or duplicate user records.

Proof of Issue

File name: backend/natgold-presale/src/lib.rs
Line number: 390

```
let kyc_status = sqlx::query_scalar!(
    r#"
    WITH inserted AS (
        INSERT INTO user_account (wallet_address)
        VALUES ($1)
        ON CONFLICT (wallet_address) DO NOTHING
        RETURNING kyc_status
    )
    SELECT kyc_status FROM inserted
    UNION ALL
    SELECT kyc_status FROM user_account WHERE wallet_address = $1
    LIMIT 1
    "#,
    &wallet_address_text
)
.fetch_one(&state.db)
.await?;

sqlx::query!(
    r#"
    INSERT INTO session (wallet_address, session_token_hash, expires_at)
    VALUES ($1, $2, to_timestamp($3))
    "#,
    &wallet_address_text,
    session_token_hash.as_slice(),
    exp as i64
)
.execute(&state.db)
.await?;
```

Severity and Impact Summary

Race conditions during concurrent wallet connections could result in database inconsistencies. While unlikely to cause security issues, it could lead to failed session creation or orphaned records requiring manual cleanup.

Recommendation

1. Wrap both database operations in a single transaction using `sqlx::Transaction`
2. Consider using a single query with CTE to perform both operations atomically
3. Add proper error handling and rollback logic for partial failures

Missing Zero Address Checks in Initialize

Finding ID: FYEO-NATP-04
Severity: **Informational**
Status: **Remediated**

Description

The `initialize()` function does not validate that critical addresses (usdt, natGold, signer, admin_, copperWallet_) are non-zero. If any of these are accidentally set to address(0) during deployment, the contract will be permanently broken and require redeployment.

Proof of Issue

File name: contract/contracts/NatGoldPresale.sol
Line number: 99

```
function initialize(
  IERC20 usdt,
  IERC20 natGold,
  address signer,
  address admin_,
  address copperWallet_,
  uint256 discount
)external initializer {
  ...

  require(discount < BPS);

  phase = Phase.Purchase;
  usdtToken = usdt;
  natGoldToken = natGold;
  kycSigner = signer;
  admin = admin_;
  copperWallet = copperWallet_;
  discountBps = discount;
}
```

Severity and Impact Summary

Deploying with zero addresses would render the contract unusable, purchases would fail, KYC verification would fail, and funds could not be transferred. However, this is a deployment-time issue easily caught, and the contract is upgradeable allowing recovery.

Recommendation

Add zero address checks for all address parameters in `initialize()`.

PAC Endpoint Ignores Expiration and Hardcodes KYC Status

Finding ID: FYEO-NATP-05
Severity: **Informational**
Status: **Acknowledged**

Description

The `/pac` endpoint has two issues when refreshing PAC using an old PAC. It doesn't validate the PAC's `expires_at` field, allowing expired PACs to be refreshed indefinitely. It hardcodes KYC status to "approved" without checking the database, allowing users to obtain fresh PACs.

Proof of Issue

File name: backend/natgold-presale/src/lib.rs
Line number: 439

```
let Json(payload) = payload.ok_or(AppError::MissingAuthorization)?;
let signed = payload.to_signed()?;
// NB! we only check the signature and ignore expires_at in PAC's payload,
//  at the same time it's checked in the smart contract
verify_pac_signature(&signed, state.pac_signer.verifying_key())
    .map_err(AppError::InvalidPacSignature)?;
let wallet_address_text = signed.address().to_string();
// We assume KYC to be already approved as user has already an older PAC
let kyc_status = "approved".to_owned();
```

Severity and Impact Summary

Users with revoked or rejected KYC can continue to refresh their PAC indefinitely using an old PAC, bypassing KYC status checks. While the smart contract validates expiration, the backend allows expired PACs to be refreshed, and assumes KYC approval without database verification.

Recommendation

Validate PAC expiration before allowing refresh, check `signed.expires_at` against current time. Query database for actual KYC status even when using old PAC for authentication. Consider removing PAC-based refresh entirely and require session token for PAC renewal.

The fund() Function Lacks Access Control

Finding ID: FYEO-NATP-06
Severity: **Informational**
Status: **Remediated**

Description

The `fund()` function has no access control restrictions, allowing anyone to call it and provide tokens from any address (via the `from` parameter). While the function requires `AwaitingFunds` phase and only accepts the exact amount needed, an attacker could front-run the legitimate funding transaction or cause the contract to transition to `Claim` phase prematurely if they have approval to transfer tokens from the specified address.

Proof of Issue

File name: contract/contracts/NatGoldPresale.sol
Line number: 246

```
function fund(address from) external {
  require(phase == Phase.AwaitingFunds, "Not awaiting funds.");

  uint256 balance = natGoldToken.balanceOf(address(this));
  uint256 needed = Math.saturatingSub(totalTokensOwed, balance);

  if (needed > 0) {
    natGoldToken.safeTransferFrom(from, address(this), needed);
  }

  phase = Phase.Claim;
  ...
```

Severity and Impact Summary

While the function is phase-gated and only transfers the exact needed amount, lack of access control could allow unauthorized parties to trigger phase transition or front-run legitimate funding. However, since only the launch team holds NatGold tokens and the function requires token approval, practical exploitation is limited.

Recommendation

Add `requireAdmin` modifier to restrict function access to authorized addresses. Consider removing the `from` parameter and requiring the caller to have pre-approved tokens.

Token Decimals Not Verified

Finding ID: FYEO-NATP-07
Severity: **Informational**
Status: **Remediated**

Description

The contract hardcodes `NAT_GOLD_DECIMALS = 18` and uses it throughout calculations without verifying that the actual NatGold token has 18 decimals. If the token implementation were to implement different decimals, all token amount calculations will be incorrect, leading to users receiving wrong amounts or the contract becoming insolvent.

Proof of Issue

File name: contract/contracts/NatGoldPresale.sol
Line number: 16

```
uint8 internal constant NAT_GOLD_DECIMALS = 18;
uint256 internal constant TOKEN_SCALE = 10 ** NAT_GOLD_DECIMALS;

// Used in transitionSettle:
priceDen = _priceDen * TOKEN_SCALE;
discountedTokenSupply = discountedTokens * TOKEN_SCALE;
discountedTokenCapPerUser = discountedTokenCap * TOKEN_SCALE;
```

Severity and Impact Summary

If NatGold token decimals differ from 18, all token calculations will be incorrect by orders of magnitude. Users could receive 1000x too many or too few tokens. However, this is unlikely as the token contract is controlled by the team and decimals are immutable once deployed.

Recommendation

Query token decimals in `initialize()` and verify it matches expected value.

Our Process

Methodology

FYEO Inc. uses the following high-level methodology when approaching engagements. They are broken up into the following phases.

Figure 2: Methodology Flow

Kickoff

The project is kicked off as the sales process has concluded. We typically set up a kickoff meeting where project stakeholders are gathered to discuss the project as well as the responsibilities of participants. During this meeting we verify the scope of the engagement and discuss the project activities. It's an opportunity for both sides to ask questions and get to know each other. By the end of the kickoff there is an understanding of the following:

- Designated points of contact
- Communication methods and frequency
- Shared documentation
- Code and/or any other artifacts necessary for project success
- Follow-up meeting schedule, such as a technical walkthrough
- Understanding of timeline and duration

Ramp-up

Ramp-up consists of the activities necessary to gain proficiency on the project. This can include the steps needed for familiarity with the codebase or technological innovation utilized. This may include, but is not limited to:

- Reviewing previous work in the area including academic papers
- Reviewing programming language constructs for specific languages
- Researching common flaws and recent technological advancements

Review

The review phase is where most of the work on the engagement is completed. This is the phase where we analyze the project for flaws and issues that impact the security posture. Depending on the project this may include an analysis of the architecture, a review of the code, and a specification matching to match the architecture to the implemented code.

In this code audit, we performed the following tasks:

1. Security analysis and architecture review of the original protocol
2. Review of the code written for the project
3. Compliance of the code with the provided technical documentation

The review for this project was performed using manual methods and utilizing the experience of the reviewer. No dynamic testing was performed, only the use of custom-built scripts and tools were used to assist the reviewer during the testing. We discuss our methodology in more detail in the following sections.

Code Safety

We analyzed the provided code, checking for issues related to the following categories:

- General code safety and susceptibility to known issues
- Poor coding practices and unsafe behavior
- Leakage of secrets or other sensitive data through memory mismanagement
- Susceptibility to misuse and system errors
- Error management and logging

This list is general and not comprehensive, meant only to give an understanding of the issues we are looking for.

Technical Specification Matching

We analyzed the provided documentation and checked that the code matches the specification. We checked for things such as:

- Proper implementation of the documented protocol phases
- Proper error handling
- Adherence to the protocol logical description

Reporting

FYEO Inc. delivers a draft report that contains an executive summary, technical details, and observations about the project.

The executive summary contains an overview of the engagement including the number of findings as well as a statement about our general risk assessment of the project. We may conclude that the overall risk is low but depending on what was assessed we may conclude that more scrutiny of the project is needed.

We report security issues identified, as well as informational findings for improvement, categorized by the following labels:

- Critical
- High
- Medium
- Low
- Informational

The technical details are aimed more at developers, describing the issues, the severity ranking and recommendations for mitigation.

As we perform the audit, we may identify issues that aren't security related, but are general best practices and steps that can be taken to lower the attack surface of the project. We will call those out as we encounter them and as time permits.

As an optional step, we can agree on the creation of a public report that can be shared and distributed with a larger audience.

Verify

After the preliminary findings have been delivered, this could be in the form of the approved communication channel or delivery of the draft report, we will verify any fixes within a window of time specified in the project. After the fixes have been verified, we will change the status of the finding in the report from open to remediated.

The output of this phase will be a final report with any mitigated findings noted.

Additional Note

It is important to note that, although we did our best in our analysis, no code audit or assessment is a guarantee of the absence of flaws. Our effort was constrained by resource and time limits along with the scope of the agreement.

While assessing the severity of the findings, we considered the impact, ease of exploitability, and the probability of attack. This is a solid baseline for severity determination.